

"ConcertCrafters". Nos han encargado diseñar e implementar un sistema para gestionar espectáculos en vivo: desde pequeños conciertos hasta grandes festivales y eventos deportivos.

Nuestro sistema almacenará información de objetos de tipo Evento.

1. Cada evento en el sistema debe tener un **identificador único** que se genere automáticamente al crearse. Necesitará un **nombre**, una **fecha** (que, una vez establecida, no debe cambiar LocalDate), un **número de entradas vendidas** y una **capacidad máxima de asistentes** (positiva).
2. El sistema debe validar que los datos iniciales sean correctos al crear un evento, lanzando **excepciones** si son inválidos:
 - Si la capacidad o el número de entradas es menor que 0
 - O si, número entradas vendidas superior a la capacidad o nombre

En el caso en el que no se pase la validación se mostrará una excepción de tipo **CraftersException** con el siguiente mensaje: *"No es posible construir un evento con esos datos"*

3. Dos eventos se considerarán iguales si tienen el mismo identificador único.
4. Los eventos tendrán un estado PENDIENTE, PROGRAMADO, CANCELADO, FINALIZADO, APLAZADO.
5. De cada evento deseamos mostrar el identificador, el nombre, la fecha y el estado.

Además deseamos tener los siguientes métodos:

- un método llamado **modificaEstado** que modifica el estado de un evento. Este método recibirá el estado nuevo al que queremos cambiar. Considera que los eventos sólo pueden avanzar a los siguientes estados, nunca retroceder. Un evento en estado pendiente puede avanzar a programado, cancelado o finalizado. Los eventos aplazados sólo pueden avanzar a estado PROGRAMADO. EL método devolverá excepción de tipo **CraftersException** si intentamos modificar a un estado no posible, con el mensaje: *"No es posible pasar al estado: XXXX"*;
- Método que **getPorcentajeOcupacion**: calcula el porcentaje de ocupación teniendo en cuenta las entradas vendidas frente a la capacidad total
- Método **getPorcentajeOcupacion (int numeroEntradas)** . Calcula el porcentaje de ocupación pero en este caso suma le suma a las entradas vendidas el numeroEntradas que recibe como parámetro.
- Todos los eventos tendrán un método abstracto llamado **calcularCosteBase** que devolverá un double.

Inicialmente, tendremos varios tipos de eventos: Concierto o Deportivo.

- Los **conciertos** un artista principal (cadena) y una tabla de tamaño 10 máximo de artistas invitados (nombre grupo o del artista)

El coste Base para los conciertos se calcula como;

$$5000 \text{ (alquiler ubicación)} + 2000 \text{ (artista principal)} + 500 * \text{artista invitado}.$$

Por ejemplo, para un evento con 3 artistas invitados sería= 5000+2000+1500

- Para los **eventos deportivos** deseamos guardar si son televisados o no.

El coste base se calculará teniendo de la siguiente manera:

$$\text{precio Alquiler estadio} - 10000 \text{ si son televisados}$$

El alquiler del estadio será 150.000 euros si la capacidad es mayor de 3000 entradas y 75000 si es menor.

Apartado 1: 1,5 puntos

Diseña y dibuja el diagrama de clases UML para este sistema

Apartado 2: 8,5 puntos

Implementa este sistema y crea una clase controladora llamada GestionaEventos que desde un método main:

- Intenta generar un evento deportivo con capacidad 500 y 600 entradas vendidas. Gestiona la excepción de manera que el programa se siga ejecutando.
- Genera un evento de tipo concierto con 200 ventas, capacidad 500 y estado APLAZADO. Deseamos mostrar el % de ocupación si se venden 50 entradas más. Muestra la información del evento. Modifica el estado para que vuelva a estar PROGRAMADO.
- Genera un evento de tipo concierto con 500 ventas y capacidad 500. Intenta añadir 20 entradas más y gestiona la excepción de forma que el programa se siga ejecutando. Muestra el coste base de este evento y su % de ocupación.
- Genera un evento deportivo con capacidad 5000, en estado CANCELADO y calcula el coste base. Muestra la información del evento. Posteriormente,
- Genera un evento deportivo con capacidad 500 y calcula el coste base. Muestra la información del evento, en estado PROGRAMADO.
- Para este último evento, intenta modificar el estado para que pase a PENDIENTE. Gestiona la excepción para que el programa finalice.

Apartado 1	Diagrama UML	1,5
	Clase Evento	1
	Método modificaEstado(Estado) con CrEx	1,5
	Método double getPorcentajeOcup()	0,75
	Método double getPorcentajeOcup(int)	0,75
	Clase Concierto: artista y invitados[] y método calculaCosteBase	1
	Clase EventoDeportivo: televisado y método calculaCosteBase	1
Apartado 2	Pruebas método main	2,5

